

Микросервисы:

Клиент (десктоп-приложение, веб-приложение, мобильное приложение – отображает пользователю картинку, обращается к API сервиса).

Организационный сервис – тот, который описывается в этой задаче. Занимается организационной частью «Космического боя» (авторизация участников, регистрация на турниры, организация матчей и пр.).

Игровой сервис – ответственен за проведение самой игры.

Кронтаб – ответственен за события, запускаемые в определённое время (посылает триггеры в организационный сервис).

Сервис рассылки уведомлений – посылает клиенту уведомление либо электронное письмо.

База данных.

Брокер сообщений (не требуется на текущий момент; но может понадобиться – см. «Узкие места»)

Endpoints

Формат: источник -> принимающий: описание эндпойнта

Клиент -> Организационный: регистрация пользователя

Клиент -> Организационный: авторизация пользователя

Клиент -> Организационный: выход пользователя

Клиент -> Организационный: создание личного матча (+ уведомление второму игроку)

Организационный -> рассылочный: отправка уведомления. Вряд ли здесь требуется набор эндпойнтов под каждый вид рассылки, достаточно единого с типом сообщения, шаблоном и приоритетностью.

Клиент -> Организационный: запуск личного матча. Организационный отправляет приказ на переадресацию к игровому сервису, после чего клиент и игровой сервис начинают общаться при помощи:

Клиент -> Игровой: универсальный эндпойнт (который мы разрабатывали на предыдущем домашнем задании); его будет достаточно для проведения и рендера космического боя.

Клиент -> Организационный: просмотреть список турниров. Турниры с высшей репутацией показываются сверху списка.

Клиент -> Организационный: подать заявку на участие в турнире (+ уведомление менеджеру турнира).

Клиент -> Организационный: принять \ отклонить заявку на участие в турнире (+ уведомление игроку - автору заявки). Отсюда же запускается перерасчёт репутации турнира, исходя из рейтинга составляющих его участников.

Клиент -> Организационный: просмотреть результаты прошедших \ активных турниров.

Кронтаб -> Организационный: выслать уведомление о скором начале боя (+ уведомление игрокам).

Игровой -> Организационный: сохранение логов боя (чтобы не выслать весь бой в конце, связь рухнет от объёма данных).

Игровой -> Организационный: завершение боя, сохранение результата (+ уведомление отыгравшим игрокам). Отсюда же запускается перерасчёт рейтинга игроков, нет никакой причины рассчитывать его отдельно. Отсюда же запускается завершение тура турнира, если была завершена последняя партия тура; оттуда запустится генерация новых пар для следующего тура с созданием новых игр (+ уведомление оставшимся в турнире игрокам), либо завершение турнира с рассылкой результатов (+ уведомление всем принявшим участие игрокам).

Клиент -> Организационный: просмотр прошедшего боя.

Клиент -> Организационный: Создание турнира.

Клиент -> Организационный: Пересоздание турнира. Создаёт уже организованный ранее турнир, что даёт буст к репутации турнира.

Кронтаб -> Организационный: Запуск турнира (+ уведомление участникам, менеджеру и организатору турнира).

Кронтаб -> Организационный: Запуск нового тура, включающее в себя запуск игр (+ уведомление участникам турнира).

Проблемы масштабирования

Проблема: увеличение количества матчей. На игровой сервис падает наибольшая нагрузка, так как сквозь него проходит наибольшее число данных, а обращаются к нему наиболее часто.

Решение: возможно увеличивать количество игровых сервисов без особых проблем. Так как соединением клиентов (игроков) занимается организационный сервис, возможно написать балансировщик, учитывающий количество текущих матчей на каждом из игровых серверов (для этого даже не потребуется обращаться к игровым сервисам – организационный всегда знает, где какие игры ведутся, так как хранит информацию о них в БД), и при создании новой игры отправлять игроков на наиболее свободный сервис.

Проблема: увеличение количества турниров. Наша текущая система отображения турниров даёт пользователю турниры с наивысшим рейтингом, которые могут быть ему не интересны. Если турниров станет слишком много, ему будет сложно найти нужные.

Решение: добавить новые фильтры \ сортировки для турниров, в которых мог бы принять участие пользователь.

Проблема: увеличение количества игроков. Как игрок будет искать друга, с которым бы хотел сразиться?

Решение: система поиска игроков.

Узкие места

Проблема: длительная передача состояния игры из игрового сервиса в организационный на длительное хранение. Много матчей, много ходов, много данных на каждом конкретном состоянии матча, они нужны в полном объёме; а веб-передача достаточно медленная.

Решение: использовать брокер сообщений для конкретно этой передачи. Передача данных через привязанный к сокетам сервис будет быстрее обычной веб-передачи, и имеющиеся брокеры легко масштабируются на любое количество подписчиков-публикаторов (игровых и организационных сервисов).

Компоненты, к которым чаще всего будут меняться требования (+ ОСР)

Турнир: организация и проведение. Будут вводиться новые системы турнира (олимпийская, с двойным вылетом, швейцарская – вместо круговой); требуется рассмотреть преждевременное завершение турнира; изменённая система начисления рейтинга, в котором очки начисляются не за отдельные матчи, а присваиваются по завершению турнира целиком; подключение, удаление или замена игроков по ходу проведения турнира, а не только до его запуска.

Рейтинг – помимо вышеописанной системы с турниром, проекту потребуются: таблица лидеров, несколько систем ведения рейтингов (например, в зависимости от формата игр).

Обе этих проблем можно решить корректной организацией неизменяемого интерфейса. Опишу ниже примеры на нативном языке (без реализации интерфейсов):

Турнир:

Действие	В текущей реализации	С потенциальными дополнениями
Добавление участника / замена участника / удаление участника	Успех, если турнир не запущен; успех без изменения состояния, если турнир запущен	Состав участников изменяется при любом состоянии турнира (для некоторых типов), что приводит к изменению генерации пар
Генерация пар нового тура	Пары определяются единственно возможным образом по круговой системе	Пары определяются согласно стратегии (система турнира). В парах могут состоять не все участники.
Начисление рейтинга	Рейтинг начисляется последовательно за матчи, сыгранные участниками	Рейтинг начисляется согласно стратегии (система турнира).
Завершить турнир	Провал, если не все туры были проведены	Успех. События завершения турнира производятся как обычно

Рейтинг игрока

Действие	В текущей реализации	С потенциальными дополнениями
Начисление рейтинга игрока в формате X	Успех, но существует только одна линейка рейтинга	Успех, изменяется соответствующий формат
Просмотр рейтинга	Получает рейтинг игрока в имеющихся форматах	Получает рейтинг игрока в имеющихся форматах P. S. Для лидербоадрa не требуется изменение этого метода; там скорее понадобилось бы получать игроков с высшим рейтингом, но это ведомость класса-репозитория (связывающегося с базой данных)

